Autonomous Option Pricing Model

What the code actually does: a tested Black–Scholes–Merton toolkit that prices hedges offline and reads live implied volatility for an always-on engine that watches markets but never trades.

Ajai Upadhyaya

ABSTRACT The options layer of the quant-trading project is a 932-line Black–Scholes–Merton module, with prices, first-order Greeks and a Brent implied-volatility solver. It is pinned by 46 tests and reproduces its reference call price of 9.4134. Two systems use it: an offline hedging overlay that prices puts, collars and put spreads at trailing realised volatility, and a live SPY volatility read that an always-on, advisory-only engine logs every 30 minutes. Re-running the code shows the pricer is accurate to about 2.5×10^{-9} in implied volatility. On matched synthetic paths, a 5%-out-of-the-money put more than halves the drawdown in a fast crash but barely changes it in a slow decline of the same size. The live read has a flaw: it solves intraday option prices against the previous close, so a 1% price move shifts measured volatility by about 4 vol points. No historical hedge results were ever saved.

01

Summary

The project's name promises an autonomous option-pricing model. What it contains is narrower and better tested than the name suggests. A single analytic pricer feeds a hedging backtest and a live volatility gauge, inside an engine that monitors and advises and is built never to act.

9.4134	2.5e−9	−28→−13%	±4.4
Pinned ATM call (S = K = 100, T = 1, σ = 20%, r = 3%); reproduced	Worst implied-vol round-trip error across 486 options	Max drawdown, unhedged vs. put-hedged, synthetic 15-day crash	Vol-point skew error from a 1% gap between live price and cached close

KEY FINDINGS

1. **One model, done carefully.** The only pricing model is analytic European Black–Scholes–Merton with a continuous dividend yield. The code has no binomial tree, Monte Carlo, Heston, local volatility or calibration. Prices, Greeks and implied volatility agree with finite differences and round trips far inside the tests' tolerances.
2. **"Autonomous" means monitoring.** The always-on loop builds a market snapshot every 45 seconds in regular trading hours, refreshes the volatility read every 30 minutes, and by its own docstring "NEVER trades, halts, or changes governance/allocation".
3. **The hedges insure gaps, not grinds.** On matched synthetic paths that fall the same 23.8%, a rolling 30-day 5%-OTM put cuts maximum drawdown from 28.1% to 13.0% when the fall takes 15 days, but only from 25.9% to 24.4% when it takes 120. In both cases the put costs about 2% a year.
4. **The live volatility read is biased intraday.** It solves live option mids against the latest cached daily close. In the week of logged reads, ATM IV ranged from 7.0% to 24.1% and moved opposite to skew (correlation −0.92), the signature a stale spot price would leave.
5. **The hedging claims are untested on history.** None of the 319 saved research experiments is a hedge run. The README's statement about drawdown, CVaR and premium drag has no saved result behind it.

02

What the system is

The quant-trading repository is a systematic ETF and equity paper-trading platform. Its options code is the `quant/options` package, 932 lines of a roughly 30,700-line codebase, built between 28 May and 4 June 2026. Everything downstream depends on three pure functions in `pricing.py`.

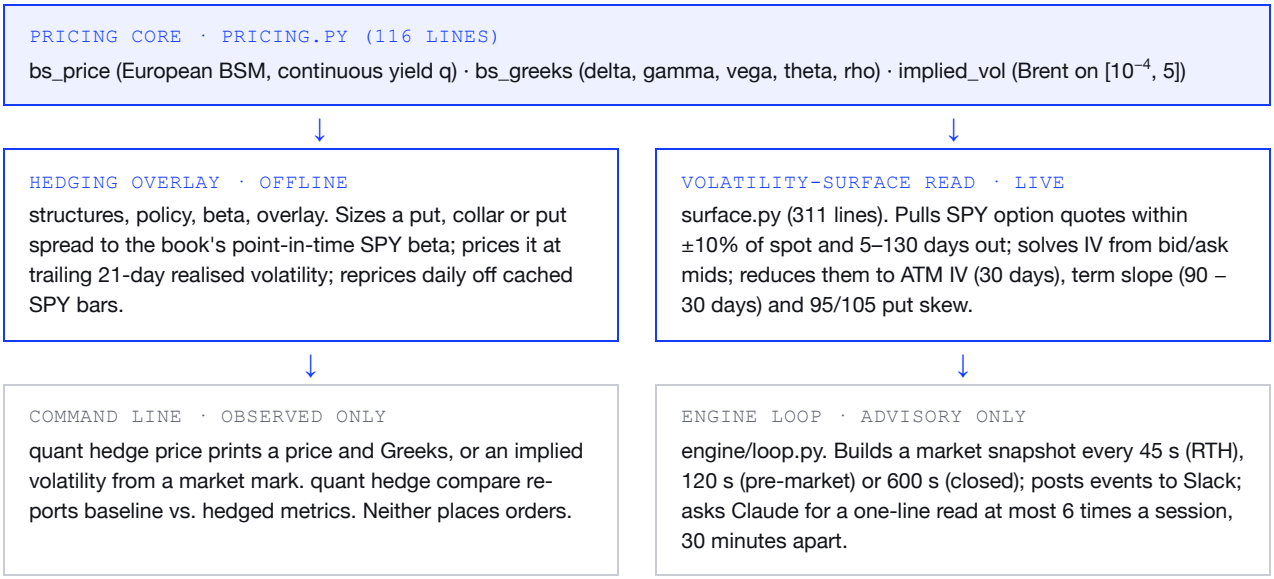


Figure 1. Data flow through the options code. Line counts and cadences from the source files; Claude limits from EngineConfig.

03

Pricing and Greeks

The pricer implements the Black–Scholes–Merton formula with a continuous dividend yield q. Time is in years, calculated as calendar days ÷ 365 at the command line and in the surface read. At or past expiry it returns intrinsic value. Non-finite or non-positive inputs return NaN instead of raising.

$$d_1 = [\ln(S/K) + (r - q + \tfrac{1}{2}\sigma^2)T] \,/\, \sigma\sqrt{T}, \quad d_2 = d_1 - \sigma\sqrt{T} \tag{1}$$

$$C = S\,e^{-qT}\,N(d_1) - K\,e^{-rT}\,N(d_2), \quad P = K\,e^{-rT}\,N(-d_2) - S\,e^{-qT}\,N(-d_1) \tag{2}$$

The Greeks are closed-form and first order only: delta, gamma, vega, theta and rho. Two conventions matter when reading output. Vega is per 1.00 of volatility, so 38.67 means 0.387 per vol point, and theta and rho are per year. Second-order sensitivities such as vanna and volga are not computed.

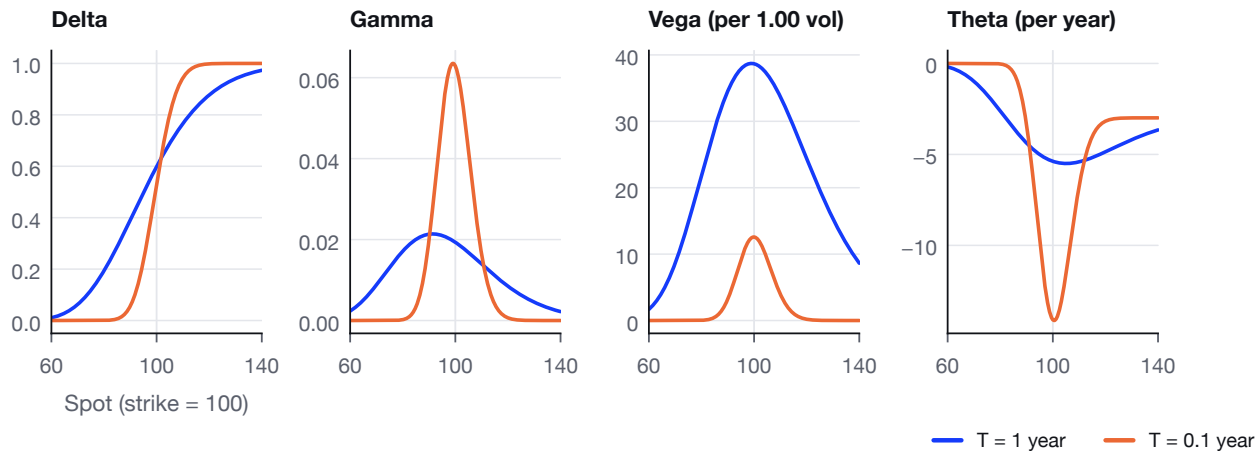


Figure 2. Call Greeks from bs_greeks against spot for one-year and 0.1-year options, same parameters. At the money, shortening maturity from 1 to 0.1 years raises gamma from 0.019 to 0.063 and deepens theta from −5.4 to −14.1 a year.

Computed for this report with quant/options/pricing.py.

Table 1. Reference values from the repository's code. The first two columns are the test case in test_pricing.py; the third is the README's worked example (spot 500, strike 475, 30 days, σ = 18%, r = 3%, q = 1.5%).

Quantity	Test case, call	Test case, put	README example, put
Price	9.4134	6.4580	2.0208
Delta	0.5987	−0.4013	−0.1481
Gamma	0.01933	0.01933	0.00896
Vega (per 1.00 vol)	38.67	38.67	33.13
Theta (per year)	−5.380	−2.469	−35.11
Rho	50.46	−46.59	−6.25
Implied vol at a mark of 6.50	—	—	29.06%

Put-call parity residual at the test case: 0.0 (tested to 10^{-9}). Textbook check at r = 5%: call 10.4506.

04

Verification

The options package carries 46 test functions across eight files. Eleven pin the pricer: one absolute price (9.4134 within 10^{-3}), put–call parity to 10^{-9}, delta, gamma and vega against central finite differences, an implied-volatility round trip, intrinsic value at expiry, and NaN on unreachable prices or invalid inputs. A property test runs 30 generated paths through the hedging overlay and checks that truncating a path never changes earlier hedged returns. That is the formal guarantee against lookahead. A project audit on 19 June 2026 recorded 1,395 tests passing across the repository with 86% coverage, and 79% for the surface module.

To see how much margin the tests leave, this report re-ran the checks over a wider range (Figure 3). At the steps the tests use, the analytic Greeks match finite differences to 6.7×10^{-12} (delta), 3.5×10^{-10} (gamma) and 1.2×10^{-7} (vega), against tolerances of 10^{-5}, 10^{-4} and 10^{-3}. The implied-volatility

solver was run on 486 out-of-the-money options: strikes 60–140, volatilities of 10%, 25% and 60%, and maturities of 30 days and one year. It recovered every input with a worst error of 2.5×10^{-9} and never failed to converge.

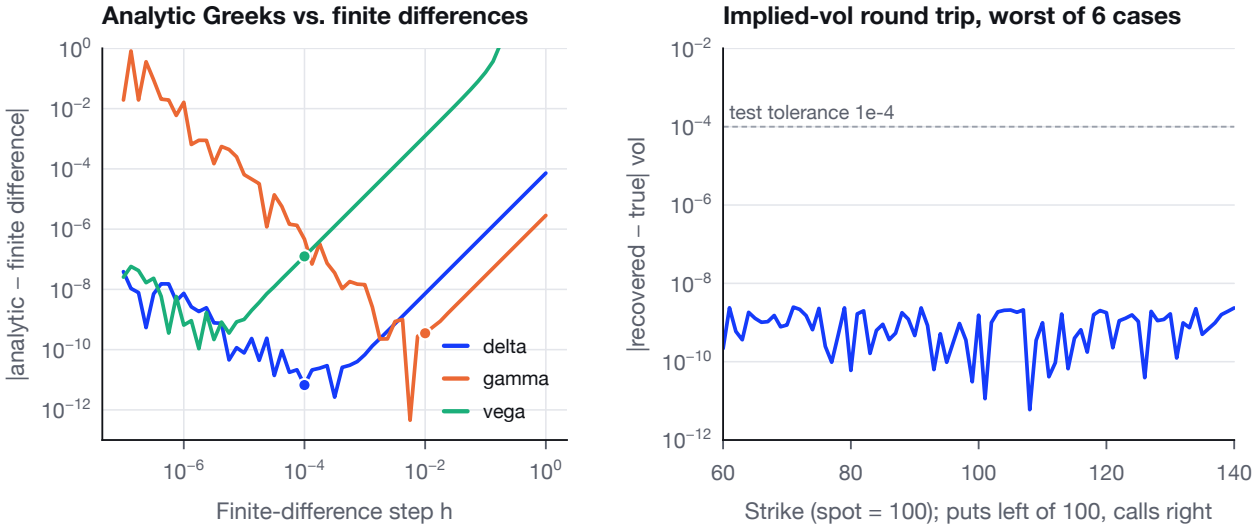


Figure 3. Left: gap between analytic Greeks and central finite differences by step size. Dots mark the steps used in the tests; the V shape is truncation error on the right and rounding error on the left. Right: the worst implied-volatility round-trip error at each strike across volatilities of 10%, 25% and 60% and maturities of 30 days and one year; the dashed line is the test tolerance.
Computed for this report with quant/options/pricing.py.

The limit of this evidence is that it is internal. Every check compares the pricer with itself: its own derivatives, its own inverse, its own parity relation. Only one absolute value is pinned, and nothing compares the code with an independent library or with market prices.

05

Implied volatility and the live surface

The solver brackets volatility between 10^{-4} and 5.0 and runs Brent's method to a tolerance of 10^{-8}. It returns NaN when a price falls outside what the model can produce. The surface reader feeds it real option mids and reduces the result to three numbers that a volatility desk watches: level, term structure and skew. Each gets a label: a regime (calm below 12%, normal below 18%, elevated below 28%, otherwise stressed), a term shape (a flat band of ±0.5 points), and a tail reading (elevated from 5 points of skew, extreme from 9).

Table 2. Surface recovery on the test suite's synthetic quotes (spot 750, 4 June 2026). Quotes are generated by bs_price at the input volatilities, then solved back.

Scenario	ATM IV in → out	Far IV in → out	Term slope	Put skew	Labels
Normal (28 / 88 days)	16.0% → 16.0%	17.0% → 17.0%	+1.0	8.0	normal · contango · elevated
Stressed (30 / 90 days)	32.0% → 32.0%	28.0% → 28.0%	−4.0	14.0	stressed · backwardation · extreme

Term slope and skew in vol points. Recovery error below 10^{-6} in every field (derived.json, surface).

Recovery is exact, but only because the reader is solving prices made by the same formula. The more informative evidence is the engine's own log. A local, uncommitted file records 2,478 engine snapshots from 5 to 11 June 2026, every one carrying a live surface reading. Because the surface refreshes every 30 minutes, they contain 68 distinct reads.

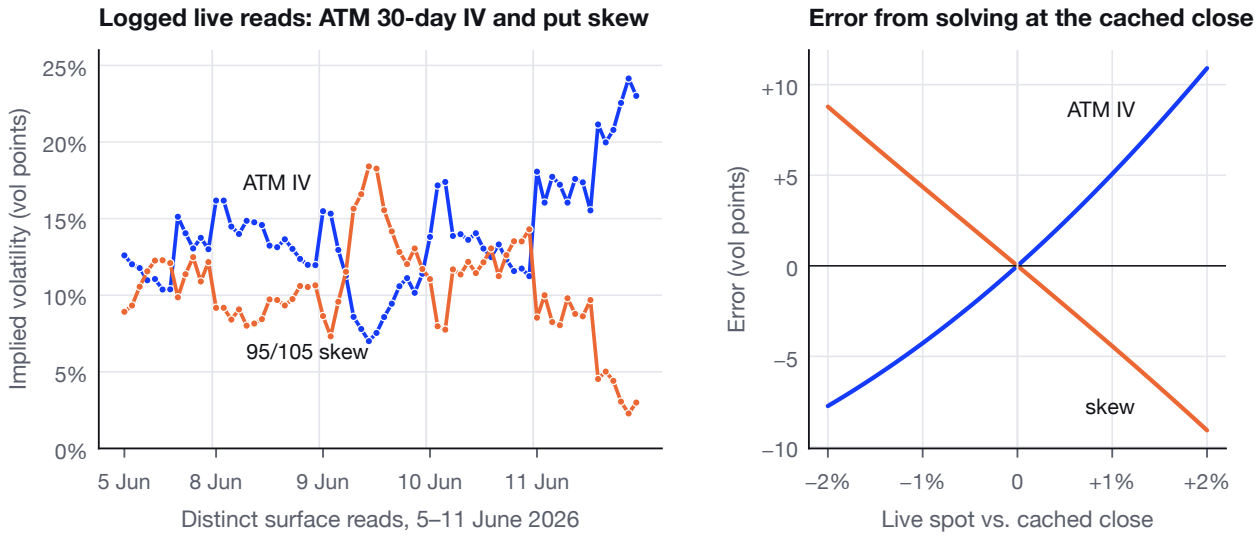


Figure 4. Left: every distinct live read the engine logged, by trading day. Right: the error the reader would make if the live price differed from the cached close, for a quote set with true ATM IV 15%, 95% put IV 20% and 105% call IV 13% (illustrative), 30 days out. Left: `quant-trading data/engine/state.jsonl (local, gitignored)`. Right: computed with `quant/options/pricing.py`.

The logged series moves too much to be a 30-day volatility. On 9 June the ATM reading fell from 15.5% at 12:10 UTC to 7.0% at 16:30 UTC while skew rose from 8.6 to 18.4 points. Across the 68 distinct reads the two series have a correlation of −0.92, and the tail label read "extreme" in 1,854 of the 2,478 snapshots. The source code explains why. `live_vol_surface` takes its spot price from the latest cached daily close, while the option quotes are live. When the market trades below the previous close, calls look cheap and puts look dear relative to that stale spot, so ATM IV falls and skew rises. Run through the repository's own solver, a 1% gap shifts ATM IV by about 4–5 vol points and skew by 4.4 points in the opposite direction, about the size of the swings in the log. That is an inference from the code and the pattern, not something proven against market data, but the fix is mechanical: solve against the live underlying price.

06

Hedging overlays

The overlay offers three index-level structures, struck on SPY and rolled every 21 trading days with a 30-day tenor. A **protective put** is struck 5% below spot. A **collar** adds a short call 5% above spot. A **put spread** buys the 95% put and sells the 85% put. The number of contracts is coverage × regime intensity × beta × book value ÷ spot. Beta is the trailing 63-day OLS slope on SPY, clamped to [0, 3]. Regime intensity is 0.25 in calm bull markets, 0.6 in choppy ones and 1.0 in a crisis. Each leg is priced by the same Black–Scholes function at trailing 21-day realised volatility, with $r = 3\%$ and $q = 1.5\%$.

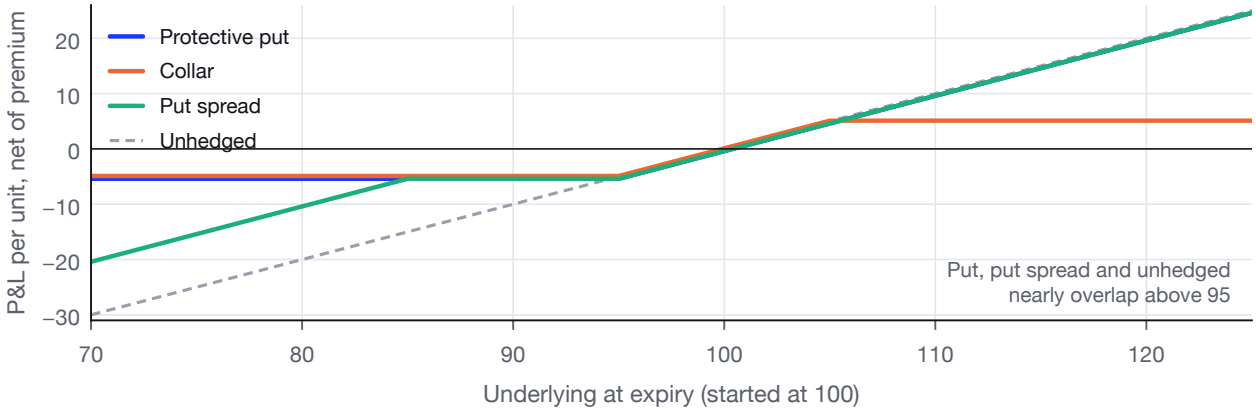


Figure 5. P&L at expiry per unit of underlying, net of premium, for the three structures built by the repository's structure builders with default settings, priced at an illustrative 18% volatility and 30 days. The put costs 0.40% of spot, the put spread 0.40%, and the collar earns a 0.11% credit.

Computed for this report with quant/options/structures.py and HedgeStructure.value.

One consequence of pricing every strike at the same volatility shows up immediately. The 85% put that the spread sells is worth almost nothing (0.403 against 0.404 for the bare put), so the spread gives up protection below 85 for no real saving. In listed markets, out-of-the-money puts usually trade at higher implied volatility than at-the-money options, which would make the spread cheaper and the bare put dearer than this model says. That is an inference about markets, not something the repository measures.

Mechanics on matched synthetic paths

No historical hedge run was ever saved, so this report ran the overlay itself on two invented paths built to isolate one variable. Both are 756 trading days of the same seeded noise, with 10% annual drift and 12.7% annualised volatility, and both fall exactly 23.8% starting on day 400. In the crash the fall takes 15 days at 2.5% daily volatility; in the grind it takes 120 days at 1.0%. The book is SPY itself, with regime scaling off and all other settings at their defaults.

Table 3. Overlay results on the two synthetic paths. Mechanics only: these are not estimates of how any hedge performs on real data.

Path / structure	Max drawdown	Total return	5% CVaR (daily)	Premium drag / yr
Crash · unhedged	−28.1%	+33.2%	−2.05%	—
Crash · protective put	−13.0%	+42.7%	−1.57%	2.4%
Crash · collar	−9.7%	+54.3%	−1.38%	−0.7%
Crash · put spread	−20.7%	+32.6%	−1.74%	2.3%
Grind · unhedged	−25.9%	+15.0%	−1.73%	—
Grind · protective put	−24.4%	+9.2%	−1.62%	2.0%
Grind · collar	−19.8%	+23.1%	−1.42%	−0.7%
Grind · put spread	−24.4%	+9.2%	−1.62%	2.0%

Premium drag is total premium ÷ years, in units of starting equity; negative means a net credit. 36 rolls per run.

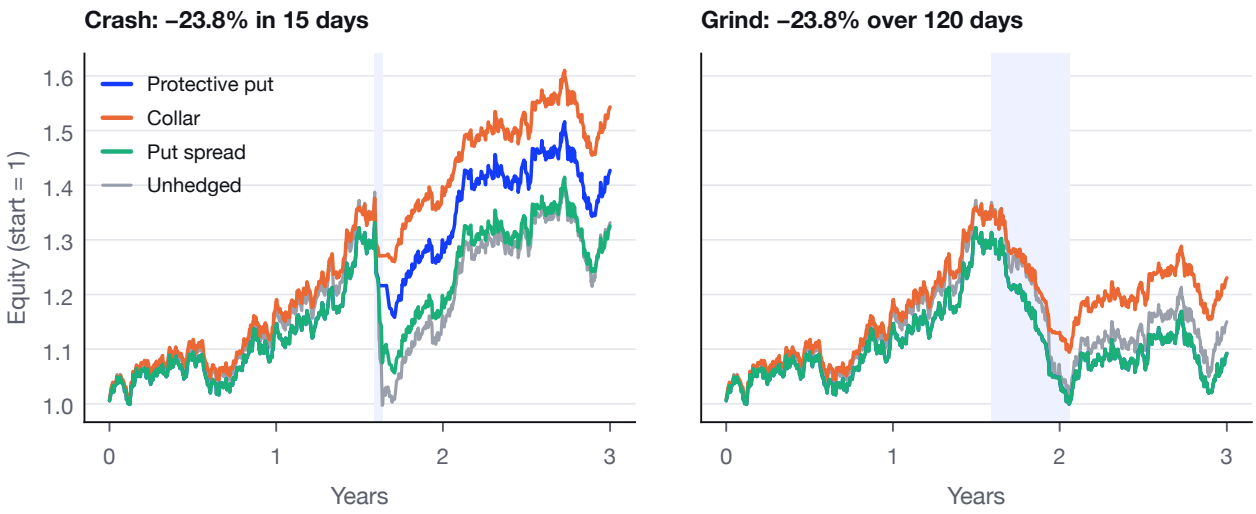


Figure 6. Equity of the unhedged book and the three hedged books. The shaded band is the decline.

Computed for this report with quant/options/overlay.py (apply_hedge, compare_hedge) on synthetic paths, seed 20260528.

The pair shows two properties of the design. First, a 30-day put struck 5% out of the money pays only when the market falls more than about 5% within one roll window. A crash clears that bar inside a single window, but a grind of the same size spreads across six windows and rarely clears it. Second, because the premium is priced at trailing realised volatility, insurance reprices after the event: the put cost 0.12% of spot per roll before the crash and 2.09% at the first roll after it, about 17 times as much. Real option markets reprice through implied volatility, which can move before realised volatility does. The collar's strong numbers on both paths come from selling upside that these particular paths happened not to need, and should not be generalised.

07

Claims against the evidence

Table 4. Options-related claims in the README and on this site, checked against the repository. The project's own plan of 19 August 2026 to "make the README true" flags the operational items.

Claim	Status	Evidence
"An analytic Black-Scholes-Merton core (price, full Greeks, implied vol)"	TRUE	pricing.py; "full" means first order. Pinned by 11 tests.
Hedging overlay: put, collar or put spread, point-in-time beta, regime-scaled, repriced daily offline	TRUE	overlay.py, policy.py, structures.py; truncation-invariance property test.
"protective puts reduce drawdown and CVaR in crashes but drag Sharpe and CAGR in calm markets"	UNEVIDENCED	No saved hedge results. Section 06 shows the mechanism on synthetic paths only.
Each hedge run logs a research experiment with drawdown and CVaR gate flags	CODE ONLY	None of the 319 records in data/research/experiments.jsonl is a hedge run.
"paper-traded on Alpaca via GitHub Actions"	OUTDATED	6 of 8 workflows disabled; live operations moved to a local machine under launchd.

Claim	Status	Evidence
"git history IS the audit trail"	OUTDATED	No commits between 20 June and the 19 August plan, so the trail stops two months short.
An "autonomous option pricing model" (this site's title)	PARTLY	Pricing is analytic and on demand. The always-on loop reads IV, but it never prices for trading, calibrates or trades.

08

Limitations

- **Model scope.** European exercise, constant volatility, rates and dividend yield, lognormal prices, no jumps and no smile. Listed SPY options are American-style, and SPY's dividend is modelled as a flat continuous yield.
- **Hedge pricing.** Hedges are priced at trailing realised volatility, flat across strikes, with no bid–ask spread, margin or assignment. Hedge P&L is added arithmetically to equity. The premium is computed over a 30/365-year tenor, but expiry is tracked as 21 trading bars, a small mismatch in time conventions.
- **Surface read.** It reads one nearest expiry and nearest strike per quantity, uses mids only, does no interpolation and no arbitrage checks, and uses a stale spot (Section 05). Its tests use synthetic quotes from the same pricer, so they check consistency, not accuracy.
- **Evidence base.** Only one absolute price is pinned, and there is no benchmark against an independent library. The engine log covers five trading days and is not committed. The overlay results in this report come from two invented paths.

A

Data and reproducibility

REPOSITORY	ajaiupadhyaya/quant-trading, commit d50a8e0 (19 Aug 2026). Options code from commits 462e-f0c (28 May 2026) to f15cfb0 (4 Jun 2026); unchanged since.
CODE READ	quant/options/ (pricing, models, structures, policy, beta, overlay, surface), quant/engine/loop.py, quant/cli.py (hedge commands), tests/options/ (46 tests).
DOCUMENTS	docs/audits/2026-06-19-honesty-audit.md; docs/superpowers/specs/2026-05-28-options-greeks-hedging-overlay-design.md; docs/superpowers/specs/2026-08-19-make-the-readme-true-design.md.
LOGGED DATA	data/engine/state.jsonl: 2,478 snapshots, 5–11 June 2026, gitignored. data/research/experiments.jsonl: 319 records, none a hedge run.
THIS REPORT	Every figure and derived number was produced by importing the repository's own modules read-only (no files, caches or environments written into it) and running them on the stated illustrative inputs. The script asserts the pinned 9.4134 before plotting. No market data was fetched; the only real-market values are the engine's logged reads.

Ajai Upadhyaya · esnupi Publications No. 07 · October 2026 · Set in Charter, Archivo Black and Courier New.